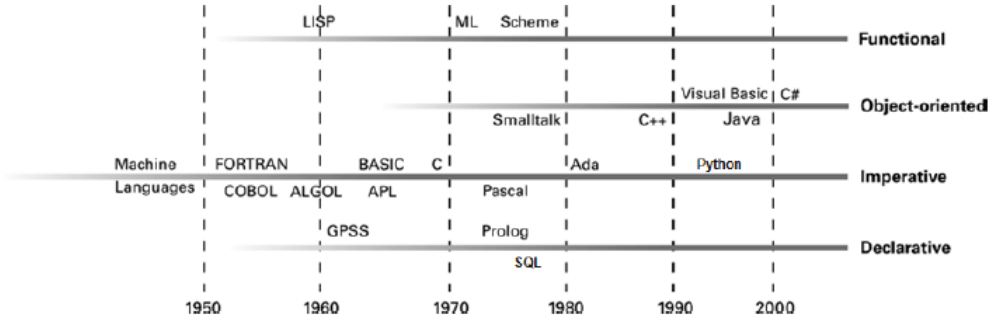
Chapter6 Programming Languages

Generation of programming languages

- **First Generation: Machine Language**: Program is encoded by number.
  - Complex and hard debugging
- **Second Generation: Assembly language**: Use a **mnemonic system** to represent instructions and memory locations.
  - Assembler: **Transfers** mnemonic instructions into the machine language instructions.
  - Programmer needs to think like the machine.
  - Essentially same to the machine language → complex

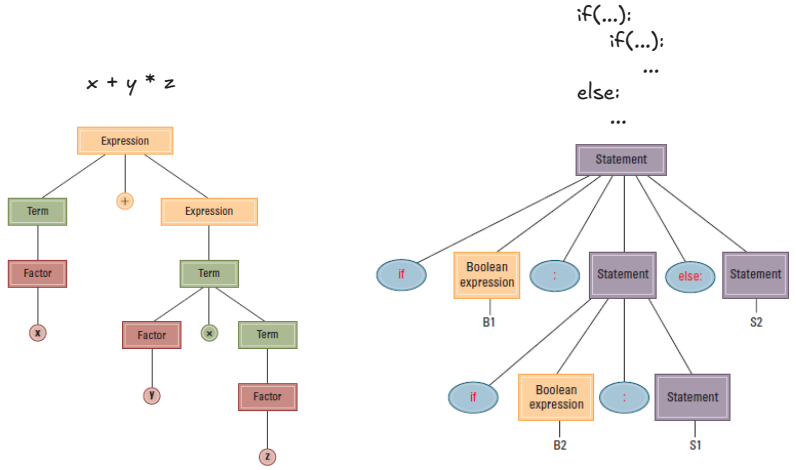
Machine Language	Assembly Language
156C	LD R5, Price
166D	LD R6, ShipCharge
5056	ADDI R0, R5 R6
30CE	ST R0, TotalCost
C000	HLT

- **Third Generation Language**: Uses high-level primitives and **focuses on logic**. `e.g. FORTRAN, COBOL, C, Java`
  - **Compiler**: Converted to machine language by a program.

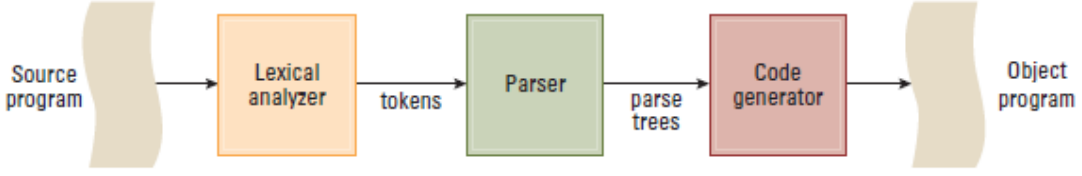


Transition Process

- **Lexical Analyzer**
  - Identify valid character strings.
  - **Token**: A series of consecutive characters forming **a single logical unit**. `e.g. i, f, ( -> if(`
- **Parser**
  - Checks each Token's content is conform to grammar and conducts them to statements.
  - Trasfers statements into **Parse Tree**.



- Code Generator
  - Transfers Parse Trees into machine language.



Classes and Objects

- **Object**: Active program unit containing both data and procedures.
- **Class**: A template from which objects are constructed
  - **Instance Variable**:Variable within an object
    - **Holds information within the object**
  - **Method**:Procedure within an object
    - Describes **the actions that the object can perform**.
  - **Constructor**: Special method used to **initialize a new object** when it is first constructed

```
#include <iostream>
#include <string>

// 1. 類別定義 (Class Definition)
// 這是所有 Laser 物件的藍圖 (Template)
class LaserClass
{
private: //外部無法使用，只有class裡的可以用
    // 實例變數 (Instance Variable) - 數據
    // 儲存每個 Laser 實例的獨特狀態
    int RemainingPower;
    std::string Name;

public:
    // 構造函數 (Constructor) - 用於初始化物件
    // C++ 推薦使用構造函數來設定初始值
    LaserClass(std::string laserName)
    {
        Name = laserName;
        RemainingPower = 100; // 初始電量為 100
        std::cout << Name << " 系統已啟動，電量: " << RemainingPower << "%" << std::endl;
    }

    // --- 方法 (Methods / Member Functions) ---

    // 瞄準向右
    void turnRight()
    {
        std::cout << Name << " 瞄準調整：向右轉動。" << std::endl;
    }
    // 瞄準向左
    void turnLeft()
    {
        std::cout << Name << " 瞄準調整：向左轉動。" << std::endl;
    }
    // 發射雷射束
    void fire()
    {
        if (RemainingPower >= 20)
        {
            RemainingPower -= 20; // 每次發射消耗 20 點電量
            std::cout << ">> " << Name << " 發射雷射！剩餘電量: " << RemainingPower << "%" << std::endl;
        }
        else
        {
            std::cout << "!! " << Name << " 無法發射：電量過低 (" << RemainingPower << "%)." << std::endl;
        }
    }
    // 獲取當前電量的方法
    int getPower()
    {
        return RemainingPower;
    }
};

// 2. 主程式 (Main Program) - 啟動與操作
int main()
{
    // A. 宣告並創建物件 (Object Instantiation)
    // Laser1 和 Laser2 是 LaserClass 的兩個獨立實例
    //Laser1. Laser2都是物件
    LaserClass Laser1("Alpha Laser");
    LaserClass Laser2("Beta Cannon");
```

```
    std::cout << "\n--- 遊戲開始：操作 Laser1 ---" << std::endl;

    // B. 調用方法 (Sending Messages) - 啟動物件行為

    // 1. Laser1 瞄準
    Laser1.turnRight();

    // 2. Laser1 發射兩次
    Laser1.fire(); // Power: 100 -> 80
    Laser1.fire(); // Power: 80 -> 60

    std::cout << "\n--- 操作 Laser2 ---" << std::endl;

    // 3. Laser2 瞄準並發射
    Laser2.turnLeft();
    Laser2.fire(); // Power: 100 -> 80

    // 4. 強行耗盡 Laser2 電量 (展示封裝和行為差異)
    Laser2.fire(); // 80 -> 60
    Laser2.fire(); // 60 -> 40
    Laser2.fire(); // 40 -> 20
    Laser2.fire(); // 20 -> 0

    std::cout << "\n--- 測試電量耗盡 ---" << std::endl;
    // 5. 再次發射：因電量不足而被阻止
    Laser2.fire();

    return 0;
}
```

- **Inheritance(繼承)**: Allows new classes to be defined in terms of previously defined classes.
- **Polymorphism(多型)**: Allows method calls to be interpreted by the object that receives the call.

```
#include <iostream>
#include <vector>
#include <string>
// 基礎類別 (父類別)
class Vehicle
{
public:
  // 虛擬函式：多型的關鍵
  virtual void honk(int volume) const {
      std::cout << "Vehicle: Basic sound at volume " << volume << "." << std::endl;
  }
  virtual ~Vehicle() {} // 確保正確的記憶體清理
};
// 衍生類別 (子類別)：繼承自 Vehicle
class Car : public Vehicle
{
public:
  // 覆寫：提供 Car 獨特的行為
  void honk(int volume) const override {
      std::string sound;
      if (volume > 5) {
          sound = "Loud BEEP! BEEP!";
      } else {
          sound = "Quiet beep.";
      }
      std::cout << "Car: " << sound << " (Volume: " << volume << ")" << std::endl;
  }
};
// 衍生類別：繼承自 Vehicle
class Truck : public Vehicle
{
public:
  // 覆寫：提供 Truck 獨特的行為
  void honk(int volume) const override {
      std::string sound;
      if (volume > 5) {
          sound = "MASSIVE HONK! HONK!";
      } else {
          sound = "Normal truck sound.";
      }
      std::cout << "Truck: " << sound << " (Volume: " << volume << ")" << std::endl;
```

```
  }
};
// 主程式：展示多型的結果
int main()
{
  // 多型集合：使用父類別指標儲存不同子類別的物件
  std::vector<Vehicle*> fleet;

  // 創建物件
  fleet.push_back(new Car());
  fleet.push_back(new Truck());

  std::cout << "--- 測試多型：單一指令，不同行為 ---" << std::endl;

  // 迴圈遍歷，多型發揮作用：運行時自動執行正確的 honk() 版本
  for (const auto& vehicle : fleet)
  {
      vehicle->honk(8); // 傳入參數 8
  }
  // 釋放記憶體
  for (const auto& vehicle : fleet)
  {
      delete vehicle;
  }
  return 0;
}
```

Programming Concurrent Activities

- **Parallel (or concurrent)processing**:simultaneous execution of multiple processes.
  - True concurrent processing requires **multiple CPUs**.
  - Can be simulated **using time-sharing** with a **single CPU**.
- **Mutual Exclusion(互斥)**: A method for ensuring that **data can be accessed by only one process at a time**.
- **Monitor**:A data item augmented with the ability to **control access to itself**

Declarative Programming

- **Resolution**: According to logic from each original statements, combines them and produces a new statement.
  - Example: `(P OR Q) AND (R OR ┌Q) :luc_arrow_right: P AND R, Q + ┌Q = none`
- **Unification**: **Assigning a value** to a variable so that two statements become **compatible**.
  - Example:

```
Rule: (Mary's lamb is at X) OR ┌(Mary is at X)
Truth: Mary is at home. -> X = home
(Mary's lamb is at home) OR ┌(Mary is at home) -> Mary's lamb is at home
```

Prolog

- **Log**ic **Pro**gramming Language
- **Fact**: A Prolog statement establishing a **fact**.
  - Consists of a single **predicate**. `e.g. parent(A,B)`
- **Rule**: A Prolog statement establishing a **general rule**.
  - `conclusion :- premise.`
  - Example:

```
wise(X) :- old(X). if X is old, X is wise too.
faster(X,Z) :- faster(X,Y), faster(Y,Z).
if X is faster than Y AND Y is faster than Z, X is faster than Y.
```